

F U N C T I O N A L ・ D E S I G N ・ S P E C I F I C A T I O N

機能設計書

トマト栽培データモニタリングシステム

文書名	機能設計書 ver. 1.0
システム名	トマト栽培データモニタリングシステム
制作者	河村 隼介 (学籍番号 18124058)
作成日	2026年6月3日
関連文書	README.md、API仕様 (Swagger UI)

改訂履歴

版数	改訂日	改訂者	改訂内容
1.0	2026/06/03	河村 隼介	初版作成。前期開発分の全機能を網羅。

目次

第1章 システム概要

1.1 目的 1.2 適用範囲 1.3 用語定義 1.4 システム構成図 1.5 技術スタック

第2章 機能一覧

第3章 機能詳細

- F-01 土壌水分の自動計測 (ESP32)
- F-02 測定の記録 (1株を3本で測り平均)
- F-03 株 (Plant) の管理
- F-04 実 (Fruit) の管理
- F-05 水やりの記録
- F-06 収穫の記録
- F-07 環境データの取得・補完
- F-08 画像の管理 (HEIC自動変換)
- F-09 ダッシュボード表示
- F-10 開花からの累積日照時間の計算

第4章 データ設計

4.1 ER概念図 4.2 テーブル定義

第5章 API設計

第6章 画面設計

第7章 外部インターフェース

第8章 非機能要件

第9章 制約事項

第1章 システム概要

1.1 目的

本システムは、トマト栽培における環境データ(土壌水分・天気・日照時間等)、および管理データ(水やり・収穫・糖度等)を自動および手動で収集し、ブラウザ上で一元的に可視化することを目的とする。最終的には、蓄積した栽培データを機械学習で分析し、糖度および収穫量を高めるための栽培条件を特定することを目指す。

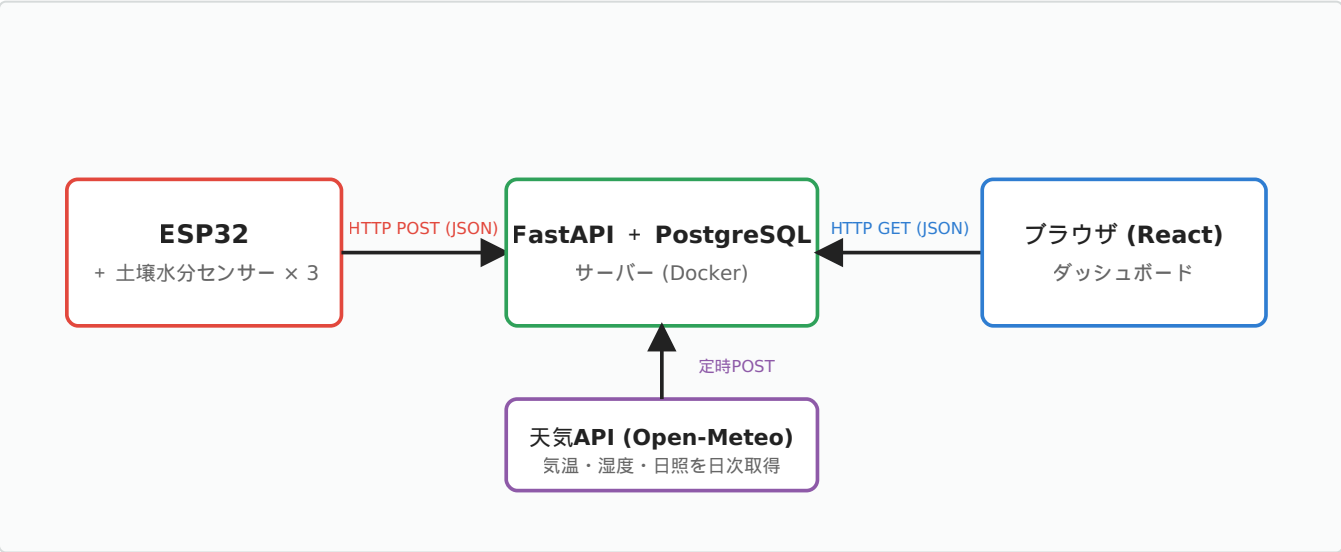
1.2 適用範囲

本書は、前期(基盤構築期)に開発済みの機能の設計を記述する。後期に予定している機械学習機能は本書の対象外とし、関連する内容は付録に記載する。

1.3 用語定義

用語	意味
株 (Plant)	1本のトマトの苗。品種・植えた日・最終丈などを記録する。
実 (Fruit)	1個のトマトの実。開花日・収穫日・糖度・サイズ等を記録する。
測定 (Measurement)	1株を3本のセンサーで測ったときの記録1回分。3本の値と平均値からなる。
センサー	静電容量式の土壌水分センサー(以下、土壌水分センサー)。
ダッシュボード	ブラウザで動作する画面全体。React で構築する。
ESP32	WiFi 機能を内蔵した小型マイコン。本システムでは計測装置として使用する。

1.4 システム構成図



計測装置の ESP32、データ蓄積を担うバックエンド(FastAPI + PostgreSQL)、表示を担うフロントエンド (React)、および外部の気象データソース(Open-Meteo) からなる。装置間の通信は全て REST API + JSON で統一する。

1.5 技術スタック

層	採用技術
マイコン	ESP32 (Freenove ESP32 WROOM) / Arduino フレームワーク (C++) / ArduinoJson / WiFi.h / HTTPClient.h
センサー	静電容量式土壌水分センサー × 3 (GPIO34・GPIO33・GPIO32)、AOUT直列に1kΩ保護抵抗
バックエンド	Python 3.12 / FastAPI / Uvicorn / SQLAlchemy 2.0 / Pydantic 2 / pydantic-settings / python-multipart / pillow / pillow-heif
データベース	PostgreSQL 16 (Docker 上で稼働)
フロントエンド	React 18 / Vite / Chart.js / react-chartjs-2 / date-fns
インフラ	Docker / Docker Compose
外部API	Open-Meteo (天気・気温・湿度・日照時間)
バージョン管理	Git / GitHub
テスト	pytest (バックエンドの測定APIに対する単体テスト)

第2章 機能一覧

機能ID	機能名	概要
F-01	土壌水分の自動計測 (ESP32)	ESP32 が3本のセンサーを読み、WiFi 経由でサーバーへ送信する。
F-02	測定の記録 (3本平均)	選択した株に対して、3本のセンサーの平均を「その株の代表値」として記録する。
F-03	株 (Plant) の管理	株の登録・編集・削除と、株ごとの履歴閲覧。
F-04	実 (Fruit) の管理	実の登録・編集・削除、並び替え、累積日照計算、写真添付。
F-05	水やりの記録	株ごとの水やり履歴(日時・量)を記録する。
F-06	収穫の記録	株ごとの収穫(日付・個数・糖度)を記録する。
F-07	環境データの取得・補完	Open-Meteo から日次で気温・湿度・日照を取得し、抜けた日も補完する。
F-08	画像の管理	画像のアップロード・一覧・削除。HEIC は自動的に JPEG へ変換する。
F-09	ダッシュボード表示	現在値・期間別グラフ・各種履歴を集約して可視化する。
F-10	累積日照時間の計算	実の開花日から現在(または収穫日)までの日照を合計する。

第3章 機能詳細

F-01

土壌水分の自動計測 (ESP32)

目的	センサーで土壌水分を測り、サーバーへ送信する。
アクター	ESP32 (装置)、サーバー (受信)
起動契機	ESP32 の通電・ディープスリープからの起床 (タイマー)

入力

- ・ センサー1〜3の生 ADC 値 (12bit, 0〜4095)。各センサーは16回サンプリングし平均を取る。
- ・ config.h に定義された DRY / WET 校正值(センサーごと)。

処理

1. 3本のセンサーを順に読み、生 ADC 値の平均を算出する。
2. 各センサーについて、`map(raw, DRY, WET, 0, 100)` で0〜100%に換算する(範囲外は丸める)。
3. WiFi に接続する(SSID・パスワードは config.h)。
4. JSON ペイロード `{device_id, readings:[{sensor_index, raw, moisture_pct}×3]}` を組み立てる。
5. サーバーの `POST /api/readings` に送信する。
6. 応答コードを確認し、シリアルヘログ出力後、ディープスリープに入る。

出力

- ・ サーバーDB の `readings` テーブルに3件(3センサー分)が登録される。

異常系

- ・ WiFi 接続失敗 → リトライ後、失敗時はそのままスリープに入る(次回起床で再試行)。
- ・ HTTP 応答コードが2xx以外 → シリアルログに記録、スリープ。

校正值はセンサー個体ごと、かつ実際に使用する土での測定値を採用する。AOUT と GPIO の間に 1kΩ の保護抵抗を必ず直列に入れる(短絡時のADC破損を防ぐ)。

F-02

測定の記録 (1株を3本で測り平均)

目的	1株を3本のセンサーで測り、その平均をその株の代表値として記録する。
アクター	運用者(ブラウザ)、ESP32 (現在値の取得)

起動契機	ブラウザ「測定の記録」パネルで株を選択し、「この株として記録」ボタンを押下。
------	--

入力

項目	必須	備考
plant_id	○	記録対象の株のID
samples[]	○	最大3本。各要素 {sensor_index, raw, moisture_pct}
note	—	任意のメモ(最大256文字)

処理

1. plant_id の株が存在するか検証する(存在しない場合は 404)。
2. 送信された samples を sensor_index ごとに raw_0/1/2、pct_0/1/2 のコラムに格納する。
3. 送信された pct の平均を計算し、avg_pct として保存する(小数1位まで)。
4. measured_at は明示指定がなければサーバー側のタイムスタンプを採用する。

出力

- measurements テーブルに1件が登録される。avg_pct がその株のその時点の代表値となる。

異常系

- plant_id が存在しない → HTTP 404
- samples が0件 → HTTP 422 (バリデーションエラー)

F-03

株 (Plant) の管理

目的	栽培しているトマトの株を識別し、関連情報(品種・植えた日・最終丈)を保持する。
アクター	運用者(ブラウザ)

サブ機能

操作	API	備考
登録	POST /api/plants	品種・植えた日を必須、最終丈・メモを任意。
一覧取得	GET /api/plants	登録順(id 降順)で返却。
取得	GET /api/plants/{id}	1件取得。
更新	PATCH /api/plants/{id}	部分更新。指定したフィールドのみ書き換え。
削除	DELETE /api/plants/{id}	関連する水やり・収穫・実は CASCADE で削除。

入力(登録)

項目	型	必須	備考
----	---	----	----

variety	文字列(64)	○	品種名
planted_date	日付	○	植え付けた日
final_plant_height_cm	数値	—	最終的な丈(cm)
note	文字列(256)	—	メモ

異常系

- 必須項目欠落 → HTTP 422
- 削除時に対象が存在しない → HTTP 404

F-04

実 (Fruit) の管理

目的	個々の実の情報(開花日・収穫日・サイズ・糖度等)を記録し、写真を添付する。
アクター	運用者(ブラウザ)

サブ機能

操作	API	備考
登録	POST /api/fruits	plant_id 必須。開花日は任意で後追記可能。
一覧取得	GET /api/fruits[?plant_id=]	plant_id で絞り込み可能。
更新	PATCH /api/fruits/{id}	収穫日・糖度・サイズ等の追記時に使用。
削除	DELETE /api/fruits/{id}	関連画像は SET NULL される。
累積日照取得	GET /api/fruits/{id}/sunlight-since-flowering	F-10を参照。

並び替え (フロント側)

- 登録順(新しい順) : id 降順
- 登録順(古い順) : id 昇順
- プランター(株)別 : plant_id でグループ化、株ごとに id 昇順

写真の追加

各実のカード内に「この実の写真」エリアを持ち、ファイル選択 → アップロードで F-08(画像管理) に fruit_id 付きでPOSTする。サムネイル表示、クリックで原寸を別タブ表示、× で削除可能。

F-05

水やりの記録

株ごとの水やりを記録する。POST /api/waterings で plant_id・watered_at(任意・省略時は現在時刻)・amount_ml(任意)・note(任意) を送信する。一覧は GET /api/waterings[?plant_id=]。削除は DELETE

`/api/waterings/{id}`。後期の機械学習で「前回の水やりからの経過時間」を特徴量化するための基礎データとなる。

F-06

収穫の記録

株ごとの収穫を記録する。`POST /api/harvests` で `plant_id`・`harvested_on`(必須)・`count`(必須)・`brix`(任意)・`note`(任意)を送信する。`brix` は本システムの「予測したい正解」のひとつであり、後期の機械学習における出力データとなる。

F-07

環境データの取得・補完

目的	天気・気温・湿度・日照時間を Open-Meteo から取得し、栽培環境データとして保存する。PC の稼働時間に左右されないよう、抜けた日もさかのぼって補完する。
アクター	運用者の PC 上の Python スクリプト (<code>scripts/fetch_weather.py</code>)、Open-Meteo (外部 API)、バックエンド(受信)
起動契機	Windows タスクスケジューラ(1日1回)、または手動実行。

処理

1. 既存の `environment_readings` を取得する (`GET /api/environments`)。
2. `note` 先頭が `auto-daily` のレコードについて、日付 → ID の辞書を作る。
3. (`--reset` 指定時)既存全件を削除する。
4. Open-Meteo の forecast API を `past_days=N`, `daily=weather_code,temperature_2m_mean,sunshine_duration` で叩く。
5. 取得した各日について、既存に存在すれば DELETE → POST(更新)、存在しなければ POST(新規)。
`recorded_at` は当該日付の12:00とする。
6. `sunshine_duration` は秒で返るため、3600で割って時間単位に変換する。

出力

- `environment_readings` テーブルに、過去N日ぶんの日次レコードが冪等に並ぶ。

異常系

- Open-Meteo に到達できない → エラー終了。次回実行で再取得される。
- サーバーがダウン → エラー終了。サーバー復旧後、再実行で完全に補完される。

F-08

画像の管理 (HEIC自動変換含む)

目的	株または実に紐づく画像をアップロード・閲覧・削除する。ブラウザが表示できない形式(HEIC)はサーバー側でJPEGに変換する。
アクター	運用者(ブラウザ)

API

操作	API
一覧	GET /api/images[?plant_id=][&fruit_id=]
アップロード	POST /api/images (multipart/form-data)
削除	DELETE /api/images/{id}
静的配信	GET /static/images/<filename>

処理 (アップロード)

1. 拡張子を取得し、許可リスト(.jpg/.jpeg/.png/.webp/.gif/.heic/.heif)に含まれるか検証する。
2. HEIC/HEIF の場合、pillow-heif を遅延 import し、`read_heif` → `to_pillow` → JPEG変換 (quality=90) を実行する。
3. UUID をファイル名にして `data/images/` に保存する。HEIC由来の場合は拡張子を .jpg に変える。
4. DB に filename・original_name・content_type・plant_id・fruit_id・description を保存する。

異常系

- 未対応拡張子 → HTTP 400
- HEIC変換ライブラリ未インストール → HTTP 400 (アプリ全体は起動を継続する Lazy Import 設計)。
- HEICデコード失敗 → HTTP 400 (例外内容を含む詳細メッセージを返す)。

F-09

ダッシュボード表示

目的	収集したデータを集約して可視化し、運用者の意思決定を支援する。
アクター	運用者(ブラウザ)
URL	http://localhost:5173 (Vite 開発サーバー)

主な構成要素

セクション	内容
SummaryCards	各センサーの最新%表示、色分け(緑/橙/赤)、データ取得時刻表示。
MeasurementPanel	株選択ドロップダウン、現在の3本値と平均、記録ボタン、選択株の測定履歴とグラフ。
EnvironmentPanel	天気・気温・湿度・日照、最新と履歴。
PlantManager	株の登録フォームと一覧。各株を展開すると水やり・収穫のSubFormと履歴。

FruitManager	実の登録フォーム、一覧、並び替え、累積日照計算ボタン、PATCH更新フォーム、写真添付。
ImageGallery	画像のアップロードフォームとサムネール一覧、ライトボックス表示。
MoistureChart	連続ログ(参考)を Chart.js でセンサー別に折れ線表示。期間切り替え(24h/7d/30d)。

F-10

累積日照時間の計算（開花日からの集計）

目的	実の生育における「光の蓄積量」を把握する。後期の機械学習における重要な特徴量となる。
API	GET /api/fruits/{id}/sunlight-since-flowering

処理

1. 対象の実の `flowering_date` を取得する。設定されていない場合は処理を中止し HTTP 404 相当のレスポンスを返す。
2. 終端日 (until) を `harvested_on`、未収穫なら現在日付とする。
3. `environment_readings` から、`recorded_at` が `[flowering_date 00:00, until 23:59]` の範囲、かつ `sunlight_hours` が NOT NULL のレコードを取得する。
4. 取得したレコードの `sunlight_hours` を合計する。
5. `{fruit_id, flowering_date, until, total_sunlight_hours, days}` を JSON で返す。

第4章 データ設計

4.1 概念モデル

本システムは8テーブルで構成される。中核は **Plant**(株) で、その配下に **Fruit**(実)、**Watering**(水やり)、**Harvest**(収穫)、**Measurement**(測定)が紐づく。Image は Plant または Fruit に紐づく。Reading(連続口グ)と EnvironmentReading(環境データ)は他テーブルとリレーションを持たない時系列データである。

関係	カーディナリティ	削除時挙動
Plant → Watering	1 : N	CASCADE
Plant → Harvest	1 : N	CASCADE
Plant → Fruit	1 : N	CASCADE
Plant → Image	1 : N (任意)	SET NULL
Fruit → Image	1 : N (任意)	SET NULL
Plant → Measurement	1 : N	CASCADE

4.2 テーブル定義

4.2.1 plants (株)

カラム	型	NULL	備考
id	INTEGER PK	NO	自動採番
variety	VARCHAR(64)	NO	品種名
planted_date	DATE	NO	植えた日
final_plant_height_cm	FLOAT	YES	最終丈
note	VARCHAR(256)	YES	メモ
created_at	TIMESTAMP WITH TZ	NO	サーバー時刻

4.2.2 fruits (実)

カラム	型	NULL	備考
id	INTEGER PK	NO	
plant_id	INTEGER FK	NO	plants.id (CASCADE)
flowering_date	DATE	YES	開花日
harvested_on	DATE	YES	収穫日

fruit_height_cm	FLOAT	YES	高さ
fruit_diameter_cm	FLOAT	YES	直径
fruit_weight_g	FLOAT	YES	重さ
final_plant_height_cm	FLOAT	YES	株の最終丈(後方互換)
brix	FLOAT	YES	糖度
note	VARCHAR(256)	YES	
created_at	TIMESTAMP WITH TZ	NO	

4.2.3 waterings (水やり)

カラム	型	NULL	備考
id	INTEGER PK	NO	
plant_id	INTEGER FK	NO	plants.id (CASCADE)
watered_at	TIMESTAMP WITH TZ	NO	省略時はサーバー時刻
amount_ml	INTEGER	YES	水の量(ml)
note	VARCHAR(256)	YES	

4.2.4 harvests (収穫)

カラム	型	NULL	備考
id	INTEGER PK	NO	
plant_id	INTEGER FK	NO	plants.id (CASCADE)
harvested_on	DATE	NO	収穫日
count	INTEGER	NO	個数 (デフォルト1)
brix	FLOAT	YES	糖度
note	VARCHAR(256)	YES	

4.2.5 measurements (測定 / 1株を3本で測った1回分)

カラム	型	NULL	備考
id	INTEGER PK	NO	
plant_id	INTEGER FK	NO	plants.id (CASCADE)
measured_at	TIMESTAMP WITH TZ	NO	サーバー時刻 (省略可)
raw_0 / raw_1 / raw_2	INTEGER	YES	各センサーの生値
pct_0 / pct_1 / pct_2	FLOAT	YES	各センサーの%

avg_pct	FLOAT	NO	提示3本の平均(小数1位)
note	VARCHAR(256)	YES	

4.2.6 readings (土壌水分 連続ログ)

カラム	型	NULL	備考
id	INTEGER PK	NO	
device_id	VARCHAR(64)	NO	送信元 (例: "esp32-01")
sensor_index	INTEGER	NO	0/1/2
raw	INTEGER	YES	生 ADC 値
moisture_pct	FLOAT	NO	変換後の%
recorded_at	TIMESTAMP WITH TZ	NO	サーバー時刻

4.2.7 environments (天候・環境)

カラム	型	NULL	備考
id	INTEGER PK	NO	
recorded_at	TIMESTAMP WITH TZ	NO	対象日の12:00など
weather	VARCHAR(32)	YES	「晴れ」「曇り」等
temperature_c	FLOAT	YES	気温(摂氏)
humidity_pct	FLOAT	YES	湿度(%) ※日次取得時は省略可
sunlight_hours	FLOAT	YES	日照時間(時間単位)
note	VARCHAR(256)	YES	"auto-daily: open-meteo @ lat,lon" 等

4.2.8 images (画像)

カラム	型	NULL	備考
id	INTEGER PK	NO	
plant_id	INTEGER FK	YES	plants.id (SET NULL)
fruit_id	INTEGER FK	YES	fruits.id (SET NULL)
filename	VARCHAR(256)	NO	UUID 由来のファイル名
original_name	VARCHAR(256)	YES	アップロード時の元名
content_type	VARCHAR(64)	YES	MIME (HEIC変換後は image/jpeg)
description	VARCHAR(256)	YES	説明
taken_at	TIMESTAMP WITH TZ	YES	撮影日時(任意)

uploaded_at	TIMESTAMP WITH TZ	NO	サーバー時刻
-------------	-------------------	----	--------

第5章 API設計

5.1 共通仕様

- ベース URL: `http://<サーバー>:8001/api`
- リクエスト/レスポンス: `application/json` (画像のみ `multipart/form-data`)
- 文字コード: UTF-8
- 認証: 前期スコープでは未実装(同一ローカルネット運用前提)
- CORS: `http://localhost:5173`、`http://localhost:5174` を許可
- エラー応答: HTTPステータス + `{detail: "..."}`

5.2 エンドポイント一覧

メソッド	パス	用途
GET	<code>/api/readings</code>	連続ログ一覧
POST	<code>/api/readings</code>	ESP32 からの連続ログ受信
DELETE	<code>/api/readings/{id}</code>	連続ログ1件削除
GET	<code>/api/measurements</code>	測定の一覧(plant_id 絞込可)
POST	<code>/api/measurements</code>	測定の記録(3本平均)
DELETE	<code>/api/measurements/{id}</code>	測定1件削除
GET	<code>/api/environments</code>	環境データ一覧
POST	<code>/api/environments</code>	環境データ登録
DELETE	<code>/api/environments/{id}</code>	環境データ削除
GET	<code>/api/plants</code>	株一覧
GET	<code>/api/plants/{id}</code>	株1件取得
POST	<code>/api/plants</code>	株登録
PATCH	<code>/api/plants/{id}</code>	株更新
DELETE	<code>/api/plants/{id}</code>	株削除
GET	<code>/api/waterings</code>	水やり一覧
POST	<code>/api/waterings</code>	水やり登録
DELETE	<code>/api/waterings/{id}</code>	水やり削除

GET	/api/harvests	収穫一覧
POST	/api/harvests	収穫登録
DELETE	/api/harvests/{id}	収穫削除
GET	/api/fruits	実一覧
GET	/api/fruits/{id}	実1件取得
POST	/api/fruits	実登録
PATCH	/api/fruits/{id}	実更新
DELETE	/api/fruits/{id}	実削除
GET	/api/fruits/{id}/sunlight-since-flowering	累積日照取得
GET	/api/images	画像一覧(plant_id / fruit_id 絞込可)
POST	/api/images	画像アップロード(multipart)
DELETE	/api/images/{id}	画像削除
GET	/static/images/{filename}	画像の配信
GET	/healthz	ヘルスチェック

5.3 リクエスト/レスポンス例(抜粋)

5.3.1 POST /api/measurements

Request:

```
{
  "plant_id": 3,
  "samples": [
    {"sensor_index": 0, "raw": 1755, "moisture_pct": 80.2},
    {"sensor_index": 1, "raw": 1760, "moisture_pct": 68.5},
    {"sensor_index": 2, "raw": 1582, "moisture_pct": 86.9}
  ],
  "note": "朝の計測"
}
```

Response 200:

```
{
  "id": 12, "plant_id": 3,
  "measured_at": "2026-05-26T07:05:50",
  "raw_0": 1755, "raw_1": 1760, "raw_2": 1582,
  "pct_0": 80.2, "pct_1": 68.5, "pct_2": 86.9,
  "avg_pct": 78.5,
}
```

```
"note": "朝の計測"  
}
```

5.3.2 POST /api/readings (ESP32 から)

Request:

```
{  
  "device_id": "esp32-01",  
  "readings": [  
    {"sensor_index": 0, "raw": 1755, "moisture_pct": 80.2},  
    {"sensor_index": 1, "raw": 1760, "moisture_pct": 68.5},  
    {"sensor_index": 2, "raw": 1582, "moisture_pct": 86.9}  
  ]  
}
```

第6章 画面設計

6.1 画面構成

本システムは Single Page Application で、ルートURL `/` に全機能が縦に並ぶ。画面は以下のセクションから成る。

セクション	役割	主な操作
ヘッダー	システム名表示	—
SummaryCards	各センサーの現在値表示	(表示のみ)
MeasurementPanel	株を選んで3本の平均を記録、選択株の履歴と推移グラフ	株選択、記録、履歴削除
EnvironmentPanel	天気・気温・湿度・日照の最新と履歴	削除
PlantManager	株の登録/編集/削除、水やり・収穫のSubFormと履歴	登録、削除、SubForm 送信
FruitManager	実の登録/編集/削除、並び替え、累積日照、写真添付	登録、更新(PATCH)、削除、並び替え、写真追加
ImageGallery	画像のアップロードと一覧、拡大表示	アップロード、ライトボックス、削除
期間切替 + MoistureChart	連続ログ(参考)の折れ線グラフ	24h/7d/30d 切替
フッター	制作者表示	—

6.2 状態と遷移

状態は各コンポーネントの React State として保持し、データ取得は `useReadings` / `useMeasurements` / `useEnvironments` / `usePlants` / `useWaterings` / `useHarvests` / `useFruits` / `useImages` の各カスタムフックで行う。各フックは `fetch` によりバックエンド API を呼び、エラーは状態として保持してUIに表示する。

6.3 ネットワーク経路

フロント (Vite, ポート5173) からバックエンド (FastAPI, ポート8001) への通信は、Vite の `server.proxy` 設定により `/api` と `/static` を中継する。CORS エラーを回避し、開発と本番でフロントエンドの呼び出しコードを変えなくてよいようにしている。

第7章 外部インターフェース

7.1 ESP32 (装置) → バックエンド

項目	仕様
プロトコル	HTTP/1.1
メソッド	POST
パス	/api/readings
Content-Type	application/json
送信頻度	config.h の INTERVAL_MS で設定(本番は900,000ms = 15分)
ネットワーク	WiFi (SSID/PWは config.h)
失敗時挙動	シリアルログにエラーを記録し、次回起床まで待機。アプリ側でのリトライキューは持たない。

7.2 Open-Meteo (外部API) → バックエンド (経由: Python スクリプト)

項目	仕様
URL	https://api.open-meteo.com/v1/forecast
パラメータ	latitude, longitude, daily=weather_code,temperature_2m_mean,sunshine_duration, past_days=N, forecast_days=1, timezone=Asia/Tokyo
レスポンス	JSON。daily.time[]・daily.sunshine_duration[]・daily.weather_code[]・daily.temperature_2m_mean[]
運用	Windows タスクスケジューラから 1 日 1 回、 <code>scripts/fetch_weather.py</code> を実行する。
失敗時	例外を投げ、終了コード1で終了。次回実行で再取得・補完される(冪等)。

第8章 非機能要件

8.1 性能

- 同時利用人数: 個人運用 (同時1ユーザー前提)
- 主要 API のレスポンス目標: 1秒以内 (ローカルネットワーク内、データ件数1万件規模)
- 連続ログのデータ量: 1日96件 (15分間隔) × 3センサー = 288件/日 (年間 約10万件 想定)

8.2 信頼性

- ESP32 の送信失敗は次回起床で自然回復する。サーバー停止中の値は失われる(リトライキューは持たない)。
- 環境データはサーバー停止していてもスクリプト次回実行時に過去N日を冪等に補完できる。
- 画像変換ライブラリの障害はアプリ全体を巻き込まない(Lazy Import で隔離)。

8.3 セキュリティ

- 同一ローカルネット内での運用を前提とし、認証機能は前期スコープ外。
- WiFi 認証情報を含む `firmware/tomato_sensor_esp32/config.h` は `.gitignore` で除外。
- 静的画像は UUID をファイル名にすることで列挙攻撃を抑止。
- HEIC のような未許可拡張子はサーバー側で拒否する。

8.4 運用

- バックエンドは Docker で起動。コード変更時は `docker compose restart backend`、依存関係変更時は `docker compose up -d --build backend`。
- DB は `pgdata` ボリュームに永続化。`docker compose down -v` でのみ初期化される。
- ログは Docker のログに集約 (`docker compose logs backend`)。
- テストは `backend/tests/` に格納。`pytest tests/test_measurements.py` で実行。

第9章 制約事項

- 本書は前期(基盤構築期)に開発した機能を対象とし、後期予定の機械学習機能は対象外とする。
- 認証・ユーザー管理は実装しない。複数ユーザーでの運用や公開ネットへの曝露は想定外。
- ESP32 と PC は同一の WiFi ネットワークに接続される前提とする。
- 校正値(DRY/WET)はセンサーごと、実際に使用する土での測定値を採用する必要がある。
- センサーの AOUT と GPIO の間には $1k\Omega$ の保護抵抗を直列に入れる。

— 本文書 終わり —